

Word AI Assistant: 面向 Microsoft Word 的 可控大语言模型写作助手系统

小组成员：陈亦乐、王语萌、徐炜翰、周煜凯

项目仓库：<https://github.com/SaLTed114/word-ai>



摘 要

大语言模型已经能够完成语法检查、学术润色、翻译和摘要等多种文本任务，但将其直接接入 Microsoft Word 等真实写作环境时，仍然面临上下文获取、结构化编辑、安全确认、多轮记忆和复合任务拆解等工程挑战。本文介绍 Word AI Assistant，一个面向 Word/WPS 写作场景的可控大语言模型写作助手系统。系统采用 FastAPI 后端、SQLite 会话存储、统一的 TaskResponse 响应结构、文档上下文构建器、可扩展 skill 文件、subagent 调度机制以及 Word Web Add-in 前端。与简单地将选中文本发送给模型的方案不同，本系统将模型输出封装为可预览、可确认、可撤销的编辑动作，并通过 agent session 和 memory 支持连续写作任务。

本文围绕系统背景、需求定义、软件架构、核心模块、运行流程、实验验证和局限性展开说明。自动化测试结果显示，当前系统的 subagent 流程、上下文构建、会话持久化、结构化 action 和多轮 mock 场景均能稳定运行。项目表明，构建可靠的 LLM 写作助手不仅依赖模型能力，也依赖围绕文档编辑场景设计的控制层、接口层和交互层。

关键词： 大语言模型，写作助手，Microsoft Word，Office 插件，结构化编辑，Subagent



目 录

第一章 引言	1
1.1 研究背景	1
1.2 问题与挑战	1
1.3 项目目标	2
1.4 主要贡献	2
第二章 相关工作与背景	5
2.1 大语言模型写作助手	5
2.2 文档中心的人机交互	5
2.3 Agent 与工具化设计	5
第三章 问题定义与结构化设计	6
3.1 任务形式化	6
3.2 统一响应结构	6
3.3 编辑动作设计	7
3.4 设计约束	8
第四章 系统总体架构	9
4.1 架构概览	9
4.2 核心模块	9
4.3 系统入口	10
4.4 处理流程	10
第五章 后端框架	11
5.1 统一启动与配置	11
5.2 API 层与服务层	12
5.3 模型网关与数据模型	12
5.4 上下文与存储	12
5.5 Prompt、Skill 与 Subagent	13
第六章 前端框架	14
6.1 插件入口与功能布局	14
6.2 运行时配置	15
6.3 文档上下文读取	15
6.4 Agent 交互与状态管理	15
6.5 Action 应用与撤销	16



第七章 实现细节	17
7.1 技术栈	17
7.2 项目结构	17
7.3 错误处理	18
7.4 安全与用户控制	18
第八章 实验与评估	20
8.1 评估目标	20
8.2 Subagent 单元测试	20
8.3 固定 Agent Flow 测试	21
8.4 多轮场景测试	21
8.5 消融式分析	21
第九章 讨论	23
9.1 有效设计	23
9.2 局限性	23
9.3 经验总结	24
第十章 未来工作	25
第十一章 结论	26
附录 A API 请求示例	27
A.1 语法修改任务	27
A.2 带文档上下文的 Agent 对话	27
A.3 显式 Subagent 调度	28
附录 B 验证命令	29
附录 C 端到端运行流程	30
致谢	31



第一章 引言

1.1 研究背景

学术写作是生物医学工程课程训练和科研实践中的重要环节。学生在撰写课程报告、实验记录、方法描述和结果分析时，需要同时满足语法准确、术语规范、语气正式和语义保真的要求。传统拼写检查或语法检查工具通常只能处理局部语言错误，而通用聊天机器人虽然能够提供更丰富的改写建议，却往往脱离用户正在编辑的文档环境，难以直接作用于 Microsoft Word 中的选区、段落或全文。

近年来，大语言模型（Large Language Model, LLM）在文本生成、语法纠错、摘要、翻译和风格改写等任务中表现出较强能力。对于普通问答场景，用户可以复制文本并等待模型返回建议；但在文档编辑场景中，文本并不是孤立输入。选中的一句话可能位于摘要、方法、结果或讨论部分，不同位置对应不同的表达目标和修改约束。因此，将 LLM 接入 Word 写作流程时，系统不仅要生成语言建议，还需要理解文档上下文并以可控方式修改文档。

此外，学术文档编辑对安全性和可追踪性有较高要求。模型不应直接覆盖用户内容，而应返回可审查的编辑建议，包括原文、替换文本、修改理由、风险等级和确认需求。对于学术写作而言，语言润色不能改变科学含义、实验条件、统计结果或术语定义。因此，一个可靠的写作助手不仅要能生成更流畅的文本，还必须让用户理解修改发生的位置、原因和影响。

1.2 问题与挑战

将大语言模型接入 Microsoft Word 等真实写作环境，需要解决若干工程问题。首先，系统需要统一处理文档上下文，包括当前选区、前后文本、文档标题、章节标题和作用范围，从而构造稳定的模型输入。其次，模型输出需要具备结构化形式。普通自然语言回答无法被前端安全应用，因此后端需要将模型结果转化为可预览、可确认、可撤销的编辑 action。

第三，写作任务通常具有多轮性。用户可能先要求解释问题，再要求更简洁版本，最后强调不得改变 scientific meaning。此时系统需要保存会话历史、写作目标、关键术语和用户偏好，而不能将每一轮请求视为孤立任务。第四，用户请求往往具有复合性质。例如“检查语法并改成更正式的学术表达”同时包



含 proofread 和 academic polish 两个目标，若全部交由单一 prompt 处理，容易造成过度改写或职责混淆。

因此，本项目关注的重点不是训练新的语言模型，而是研究如何将通用 LLM 封装为可控、可扩展、可嵌入 Word 的写作辅助系统。系统需要在模型能力之外提供上下文构建、结构化响应、会话记忆、任务拆解、前端预览和撤销机制，使模型输出能够可靠进入文档编辑 workflow。

1.3 项目目标

基于上述需求，本文设计并实现 Word AI Assistant，一个面向 Word/WPS 写作场景的可控大语言模型写作助手系统。项目从“给定文本、固定 prompt、模型返回结果”的简单 baseline 出发，逐步加入上下文构建、结构化响应、会话记忆、subagent 调度、skill 管理和 Word 插件集成，形成完整的软件框架。

系统的核心目标包括三个方面。第一，提供统一的后端 API，使 Word 插件、命令行调试入口和简单 Web demo 能够共享同一套任务执行逻辑。第二，设计统一的 TaskResponse 和 TextAction schema，将模型输出转化为可落地的文档编辑动作。第三，引入 agent session、session memory 和 subagent 调度机制，使系统能够支持多轮写作任务和复合任务拆解。

1.4 主要贡献

本文的主要贡献可以概括为以下几点：

1. 提出并实现了一个面向 Word 文档编辑的 LLM 写作助手架构，覆盖后端 API、模型调用、会话存储和前端插件。
2. 设计统一的 TaskResponse 和 TextAction schema，将模型输出封装为可预览、可确认、可撤销的文档编辑动作。
3. 实现文档上下文构建器和 agent session 机制，使系统能够在多轮写作中维护上下文、历史和用户偏好。
4. 设计并验证 subagent 调度框架，支持 proofread、academic polish、summarize、translate 和 formula 等子任务的拆分与合并。

图 1.1 至图 1.3 展示了系统在用户界面、skill 管理和 subagent 调度方面的主要实现效果。



图 1.1 系统整体用户界面

Figure 1.1 Overall user interface of the system

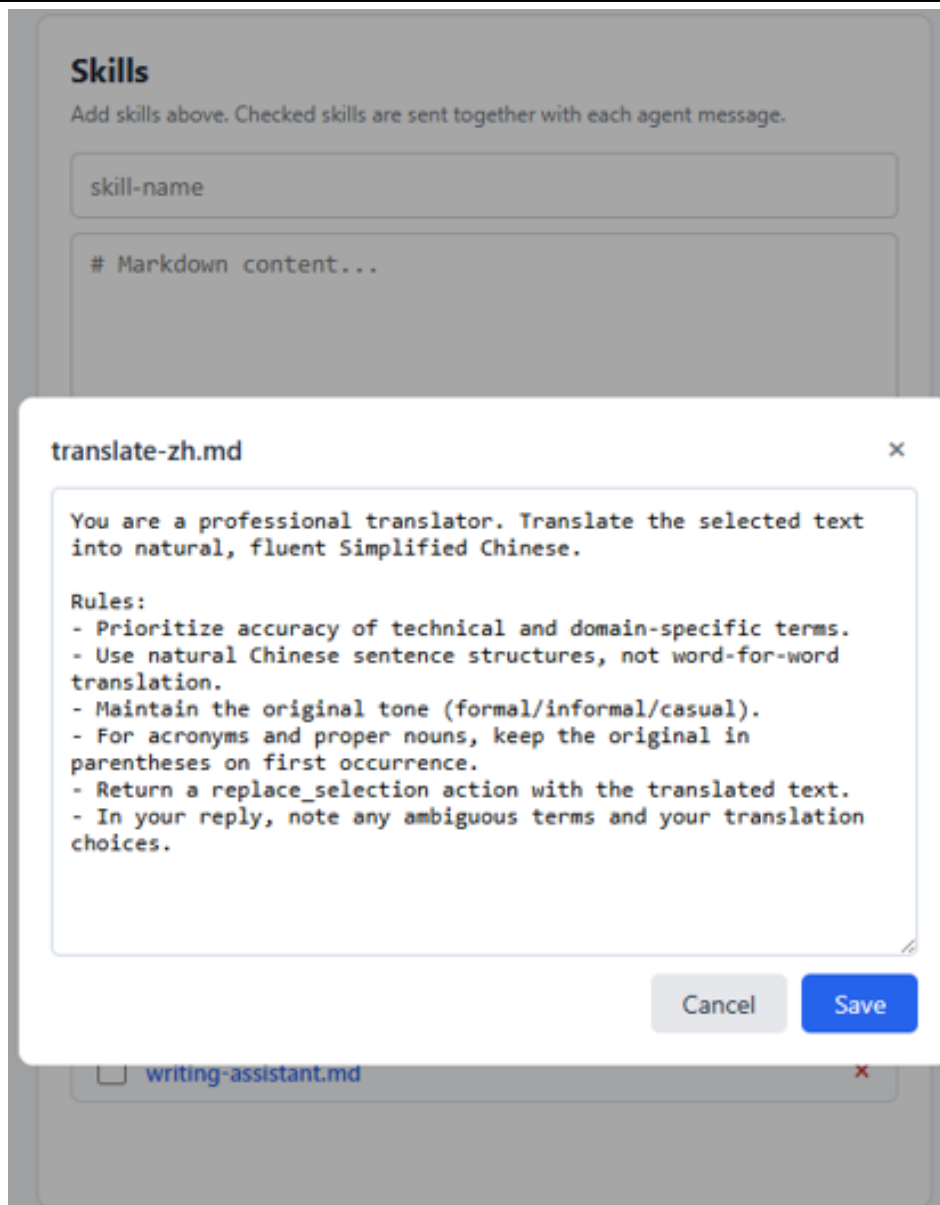


图 1.2 Skill 功能界面

Figure 1.2 Skill management interface



图 1.3 Subagent 功能界面

Figure 1.3 Subagent feature interface



第二章 相关工作与背景

2.1 大语言模型写作助手

大语言模型写作助手通常以自然语言输入和自然语言输出为主要交互形式，能够完成语法纠错、文本改写、摘要生成、翻译和问答等任务。对于普通聊天场景，这种模式已经能够满足基本需求；但在文档编辑场景中，系统不仅需要回答“如何修改”，还需要明确“修改哪里、如何应用、是否存在风险、是否需要用户确认”。因此，本项目关注的重点不是单个 `prompt` 的生成质量，而是大语言模型在写作软件中的系统封装方式。

2.2 文档中心的人机交互

Microsoft Word 等文档编辑器与普通聊天窗口存在显著差异。文档具有标题、段落、选区、公式、批注和上下文等结构，用户操作也不是一次性问答，而是“选择文本、查看建议、预览修改、应用或撤销、继续追问”的连续过程。若 AI 输出只能以普通文本形式返回，用户仍需手动复制粘贴，系统难以真正融入写作流程。面向文档的写作助手需要将模型输出转换为可执行的编辑动作，并在界面层提供确认和回退机制。

2.3 Agent 与工具化设计

Agent 式系统强调上下文维护、任务分解和工具调用。本项目中的 `agent` 并非完全自主的规划系统，而是一个轻量级写作 `agent`：它读取当前选区、会话历史和显式 `memory`，判断用户意图属于解释、改写、润色还是结构化编辑，并返回统一的 `TaskResponse`。在此基础上，`subagent` 机制将复合任务拆分为 `proofread`、`academic polish`、`summarize`、`translate` 和 `formula` 等专门子任务，从而降低单一 `prompt` 的职责混杂，提高系统行为的可解释性和可测试性。



第三章 问题定义与结构化设计

3.1 任务形式化

本文将 Word AI Assistant 的核心任务定义为结构化文档写作辅助任务。给定用户当前请求、Word 文档选区、上下文窗口、会话历史和可选的 skill 或 sub-agent 配置，系统需要判断用户意图，并输出一组可被前端安全执行的编辑建议。与普通聊天机器人不同，本系统的目标不是仅生成一段自然语言回答，而是把模型结果转化为可预览、可确认、可撤销的文档操作。

一次任务的主要输入如表 3.1 所示。

表 3.1 结构化写作任务的主要输入

Table 3.1 Main inputs of the structured writing task

输入项	作用
message	用户自然语言请求，如润色、翻译、解释或公式处理
selection	Word 中当前选中文本或目标文档范围
document_context	文档标题、章节标题、全文、选区位置和前后上下文
history	当前 agent session 中的多轮对话历史
memory	文档摘要、写作目标、关键术语和用户偏好
skills/subagents	可选 skill 或子任务配置

3.2 统一响应结构

系统输出采用统一的 TaskResponse schema。该结构将“面向用户的解释”和“面向 Word 的编辑动作”分离，使前端不需要从自由文本中猜测模型意图。其核心字段如下：

```
1 {  
  "task": "syntax | word_choice | style | formula | agent",  
3  "reply": "user-facing explanation",  
  "summary": "short summary or null",  
5  "actions": [],  
  "final_text": "rewritten text or null",  
7  "subagent_calls": []  
}
```



各字段含义见表 3.2。

表 3.2 TaskResponse 的核心字段

Table 3.2 Core fields of TaskResponse

字段	设计含义
task	标记任务类型，便于前端选择展示方式
reply	向用户解释修改原因、风险或后续建议
summary	对本轮结果进行简短概括，可用于历史记录
actions	可执行编辑动作列表，是系统落地到 Word 的关键
final_text	当任务需要整体改写时返回完整文本
subagent_calls	记录本轮调用的子任务，便于调试和展示

3.3 编辑动作设计

actions 字段是系统区别于普通问答式写作助手的核心。每个 **action** 都描述一次具体的文档操作，包括操作类型、目标范围、原文、替换文本、预览内容、修改理由、风险等级和确认要求。一个典型的替换动作如下：

```
{
  "type": "replace_selection",
  "target": "selection",
  "original": "This method is good and useful.",
  "replacement": "This method is effective for the proposed workflow.",
  "reason": "improve academic tone",
  "risk_level": "low",
  "requires_confirmation": true
}
```

常见 **action** 类型如表 3.3 所示。



表 3.3 常见编辑动作类型

Table 3.3 Common edit action types

类型	用途
replace_selection	替换当前 Word 选区
replace_range	替换指定范围文本，适合更细粒度定位
add_comment	不直接修改文本，只添加解释或风险提示
insert_equation	在光标处插入公式或替换选中公式
ask_user	当语义不确定时请求用户补充信息

3.4 设计约束

该形式化设计服务于三个约束。第一，模型输出必须能被程序稳定解析，因此所有任务共享同一响应结构，并由 Pydantic 模型验证。第二，任何文档修改都必须保留原文和替换文本，前端只能在用户确认后应用修改。第三，复杂请求可以被拆解为多个 subagent 调用，例如先由 proofread 处理语法问题，再由 academic polish 改善学术表达，最后合并为统一的 TaskResponse。

通过这种设计，Word AI Assistant 将 LLM 从“生成文本的聊天模型”封装为“产生受控编辑动作的文档助手”。这也是后续系统架构、前端交互和自动化测试设计的基础。



第四章 系统总体架构

4.1 架构概览

Word AI Assistant 采用前后端分离、后端统一调度的设计。系统以 FastAPI 后端为核心，将 Word 插件、简单 Web 演示页面和命令行调试入口统一连接到同一套任务 API、prompt 资源、skill 资源、模型调用接口和会话存储。整体架构如图 4.1 所示。

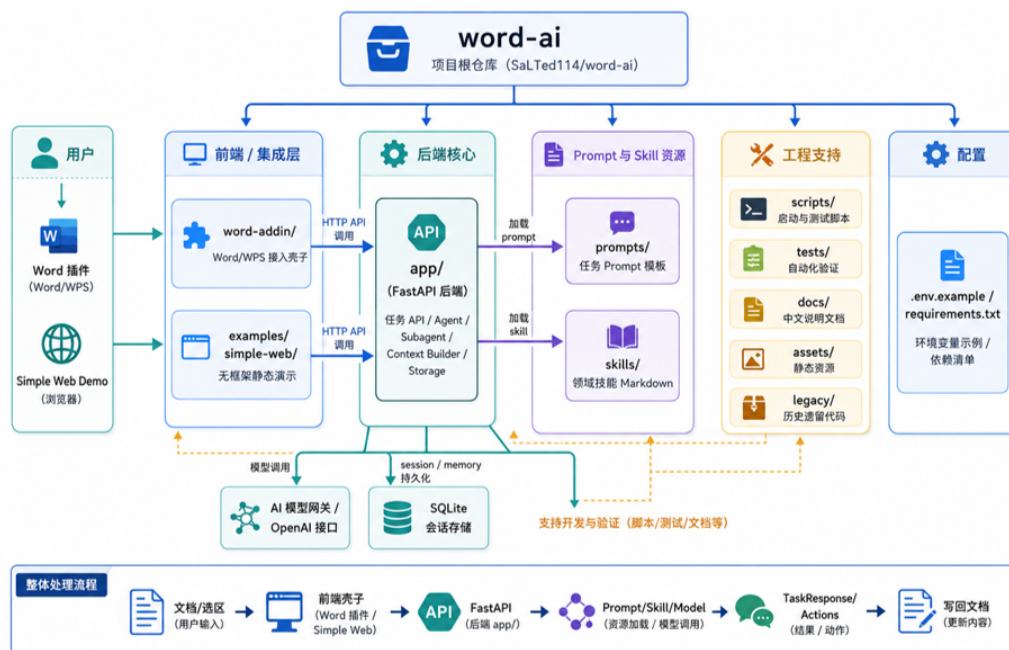


图 4.1 Word AI Assistant 系统总体架构

Figure 4.1 Overall architecture of Word AI Assistant

从用户侧看，系统支持两类主要使用方式：一种是在 Microsoft Word 或 WPS 中通过 Web Add-in 直接处理文档选区；另一种是在浏览器中通过 simple web demo 验证同一套后端能力。两类前端都通过 HTTP API 调用后端服务，因此文档上下文构建、模型调用、结构化响应和会话管理逻辑不会分散在多个客户端中。

4.2 核心模块

系统后端由 app/ 包实现，承担任务 API、agent、subagent、context builder 和 storage 等职责。Prompt 与 skill 资源以 Markdown 文件形式存放，运行时由后端



加载并注入模型输入。模型调用层通过 OpenAI-compatible 接口或 direct endpoint 与外部 LLM 服务通信；会话和 memory 则通过 SQLite 持久化，用于支持多轮写作任务。

前端主要由 word-addin/ 和 examples/simple-web/ 组成。Word 插件负责读取选区、发送 document context、展示 TaskResponse 和应用 action；simple web demo 则用于在不打开 Word 的情况下验证核心流程。工程支持部分包括启动脚本、测试脚本、文档资源和历史兼容代码，用于支撑开发、验证和打包。

4.3 系统入口

系统保留多个入口，是为了降低调试和演示成本。不同入口共享后端核心逻辑，差异仅在于用户界面和上下文来源。表 4.1 总结了主要入口。

表 4.1 系统主要入口

Table 4.1 Main system entry points

入口	主要用途	共享能力
Word Add-in	在 Word/WPS 中处理真实文档选区	API、prompt、schema、session
Simple Web Demo	浏览器中演示和调试写作流程	API、prompt、schema、session
HTTP CLI / Scripts	自动化测试和接口验证	API、context builder、storage

4.4 处理流程

一次典型请求从用户选择文本开始。前端读取选区和必要的文档上下文后，将其发送给后端；后端调用 context builder 生成标准化 TextRequest，再根据任务类型选择 direct task、agent 或 subagent 流程。模型输出被解析并校验为 TaskResponse，其中的 reply 用于解释结果，actions 用于描述可执行编辑。最后，前端将 action 渲染为预览项，用户确认后再写回文档。

这种架构的核心优势在于控制边界清晰：前端负责用户交互和文档操作，后端负责任务调度和模型封装，模型只负责生成受 schema 约束的建议。系统因此能够在利用 LLM 语言能力的同时，维持对文档修改过程的可审查性和可撤销性。



第五章 后端框架

5.1 统一启动与配置

系统后端以 `server.py` 作为统一启动入口。该入口在本地同时启动 FastAPI 后端和 Word Add-in 静态文件服务，使插件页面、设置页面和后端 API 能够在同一应用中协同工作。启动时，程序会读取命令行参数或环境变量，确定 API 端口、插件页面端口和资源目录；在打包环境下，还会根据运行位置查找证书、静态资源和配置文件。

配置层由 `app/config.py` 负责，主要管理模型 APIkey、模型名称、OpenAI-compatible base URL、direct endpoint、代理设置和 JSON mode 开关。源码运行时，配置通常来自项目目录下的 `.env`；打包后则优先使用用户可写的数据目录。这样可以避免把密钥和用户设置写入只读安装目录。

图 5.1 展示了后端框架中 API 层、服务层、模型网关、上下文构建、存储和 subagent 模块之间的关系。

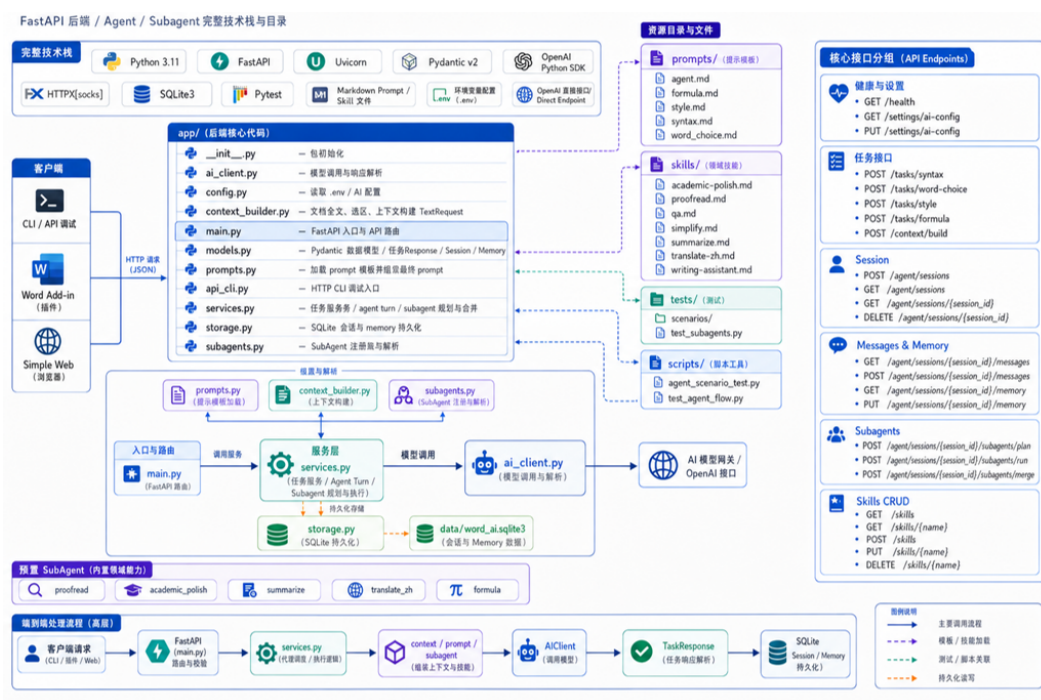


图 5.1 后端框架图

Figure 5.1 Backend framework diagram



5.2 API 层与服务层

`app/main.py` 是 FastAPI 应用入口，负责注册 HTTP 接口、初始化模型客户端和创建会话存储。API 层只处理请求校验、路由分发和异常转换，具体任务逻辑交给服务层完成。主要接口覆盖健康检查、模型配置、语法检查、用词优化、风格改写、公式处理、上下文构建、agent session、subagent 调度和 skill 管理。

`app/services.py` 是业务逻辑核心。它根据任务类型选择对应 prompt，调用模型网关，并将模型输出解析为统一的 `TaskResponse`。对于普通任务，服务层直接运行 `syntax`、`word choice`、`style` 或 `formula` 流程；对于多轮任务，服务层会读取 `session history` 和 `memory`，构建 `agent prompt`；对于复合任务，则调用 `subagent planner`、单个 `subagent` 或结果合并逻辑。

5.3 模型网关与数据模型

模型调用由 `app/ai_client.py` 封装。系统支持两种调用方式：一种是使用 OpenAI-compatible SDK，通过 `base_url`、`api_key` 和 `model` 访问模型；另一种是 `direct endpoint` 模式，适配学校或本地网关提供的 HTTP 接口。模型网关会设置较低 `temperature`，并在可用时启用 `JSON mode`，以降低输出格式不稳定的风险。

所有跨模块数据结构定义在 `app/models.py` 中。核心模型包括文本请求、上下文、编辑动作、任务响应、会话记忆和 agent 规划等对象。这些模型使前端、服务层、存储层和测试代码共享同一套数据约束，避免不同模块各自解释模型输出。

5.4 上下文与存储

上下文构建器位于 `app/context_builder.py`。它将前端传入的 `document context` 转换为标准 `TextRequest`，处理 `selection offset`、选区文本匹配、`active scope` 和 `before/after` 上下文截取。该模块的作用是把 Word 插件、Web demo 和 CLI 的不同输入形式统一成稳定的模型输入。

本地持久化由 `app/storage.py` 负责，底层使用 SQLite。系统保存三类核心数据：agent session、历史消息和 session memory。表 5.1 总结了主要存储对象。



表 5.1 本地存储对象

Table 5.1 Local storage objects

对象	作用
agent_sessions	保存会话 ID、标题、创建时间和更新时间
agent_messages	保存每轮 user/assistant 消息及响应 JSON
agent_session_memory	保存文档摘要、写作目标、术语和偏好

5.5 Prompt、Skill 与 Subagent

Prompt 文件位于 `prompts/`，用于定义基础任务规则，例如 agent、syntax、word choice、style 和 formula。Skill 文件位于 `skills/`，用于描述可运行时组合的专门能力，例如 proofread、academic polish、summarize、translate 和 formula。二者的区别在于：prompt 是任务类型的基础约束，skill 是可插拔的领域指令。

Subagent 注册表由 `app/subagents.py` 管理。每个 subagent 包含名称、默认 skill、简短 instruction、允许的 action 类型和上下文策略。表 5.2 给出系统中的典型 subagent。

表 5.2 典型 subagent 设计

Table 5.2 Typical subagent design

Subagent	默认 skill	主要职责
proofread	proofread.md	语法、拼写和清晰度检查
academic_polish	academic-polish.md	学术表达润色
summarize	summarize.md	摘要和概括
translate_zh	translate-zh.md	中英文翻译
formula	formula.md	公式和 LaTeX 处理

通过上述框架，系统将 API、模型调用、上下文构建、会话存储和任务拆解分离到不同模块中。这样的设计降低了 Word 前端与模型实现之间的耦合，也使后续测试、调试和功能扩展更容易进行。



第六章 前端框架

6.1 插件入口与功能布局

Word Add-in 前端位于 `word-addin/` 目录，核心入口由 `manifest.xml` 声明。插件使用 Word Web Add-in 机制嵌入 Microsoft Word 或 WPS，获得读取和写回文档的能力。Ribbon 中提供语法检查、用词优化、风格改写、公式处理、Agent、Skills 和 Settings 等入口，其中前几类适合处理当前选区，Agent task pane 则用于多轮写作任务。

前端界面采用任务窗格形式。用户在 Word 中选中文本后，可以直接发起单轮任务，也可以在 task pane 中输入更复杂的自然语言请求。前端不会自行生成模型 prompt，而是读取文档上下文并调用后端 API，由后端完成任务判断、prompt 构建和模型调用。

图 6.1 展示了 Word Add-in 前端的主要结构，包括 Ribbon 入口、任务窗格、设置页面、运行时配置、Office.js 文档访问和后端 API 调用。



图 6.1 前端框架图

Figure 6.1 Frontend framework diagram



6.2 运行时配置

Word 插件页面通过本地 HTTPS 静态服务加载。由于后端 API 端口可能因占用而变化，系统在启动时动态生成 `/runtime-config.json`，前端通过共享脚本读取 API base URL。这样，打包后的应用不需要在 JavaScript 中写死端口，也便于在开发和演示环境中切换服务地址。

Settings 页面同时负责模型配置和本地 UI 行为配置。模型相关配置包括 base URL、direct endpoint、model、JSON mode 和 API key 等；UI 行为配置包括是否显示 action details、是否显示 undo/review、是否启用 auto subagents、subagent 执行模式以及 memory 文本等。这些设置使前端能够在不修改源码的情况下调整系统行为。

6.3 文档上下文读取

前端通过 Office.js 读取当前文档。典型流程包括：首先获取当前选区，再读取文档正文；如果存在选区，则以前端选区作为主要目标文本；如果没有选区，则退化为全文任务。前端还会在本地截取选区前后文本，用于预览和调试。

真正的上下文标准化仍由后端 `/context/build` 完成。前端只负责发送原始 document context，包括标题、章节标题、全文、selection、active scope 和上下文窗口大小。这样可以避免 Word 插件、Web demo 和 CLI 分别实现不同的选区解析逻辑。

6.4 Agent 交互与状态管理

Task pane 维护多轮 agent 交互所需的前端状态，例如当前 session ID、是否正在发送请求、历史记录是否加载完成、最近一次应用的 action 以及当前错误信息。用户发送消息时，前端会创建或复用 agent session，并将 message、document context 和本地 memory 一并发送到后端。

后端返回 TaskResponse 后，前端将 reply 渲染为对话内容，将 summary 用于结果概括，并把 actions 渲染为可预览的编辑建议。这样的设计使聊天展示和文档修改保持分离，用户可以先阅读解释，再决定是否应用修改。



6.5 Action 应用与撤销

前端只根据结构化 action 修改 Word 文档。常见的 `replace_selection` 会替换当前选区，`add_comment` 用于提示用户检查某段内容，`insert_equation` 用于插入或替换公式。对于需要确认的 action，前端先展示原文、替换文本和修改理由，用户确认后才写回文档。

为了降低误操作风险，前端保存最近一次应用的修改信息，用于 `undo` 和 `review`。Undo 可以将原始文本写回文档，Review 则用于展示 before/after 对比。该机制与后端的 `requires_confirmation` 和 `risk_level` 字段共同构成可控编辑流程。



第七章 实现细节

7.1 技术栈

本项目采用 Python 后端与 Office Web Add-in 前端结合的实现方式。后端使用 FastAPI 提供 HTTP API，使用 Pydantic 定义请求和响应 schema，并通过 SQLite 保存本地会话数据。前端基于 Office.js 与 Word 文档交互，负责读取选区、展示建议和写回用户确认后的修改。表 7.1 总结了系统主要技术栈。

表 7.1 项目核心技术栈

Table 7.1 Core technology stack

层次	技术
后端框架	FastAPI
ASGI 服务	Uvicorn
数据校验	Pydantic v2
HTTP 客户端	httpx
模型接口	OpenAI-compatible SDK 或 direct endpoint
本地存储	SQLite
Word 集成	Office.js Word Web Add-in
打包发布	PyInstaller 相关脚本
测试	pytest、FastAPI TestClient、mock AI client

这种技术组合的优点是实现成本较低、模块边界清晰，并且适合本地开发型写作工具。FastAPI 和 Pydantic 负责接口和 schema 约束，Office.js 负责 Word 文档操作，SQLite 负责轻量级状态持久化，测试代码则用于验证关键工程路径。

7.2 项目结构

项目目录围绕后端核心、前端插件、prompt/skill 资源和工程脚本进行划分。后端主要集中在 `app/` 包中，前端插件位于 `word-addin/`，测试与验证脚本位于 `tests/` 和 `scripts/`。主要目录结构如下：

```
1 app/  
  main.py  
3 services.py  
  ai_client.py
```



```
5 context_builder.py
   models.py
7 storage.py
   prompts.py
9 subagents.py

11 prompts /
   skills /
13 word-addin /
   scripts /
15 tests /
   examples / simple-web /
17 server.py
```

其中，`main.py` 负责 API 路由，`services.py` 负责业务流程，`ai_client.py` 负责模型调用，`context_builder.py` 负责文档上下文标准化，`models.py` 负责共享数据结构，`storage.py` 负责 SQLite 存储，`subagents.py` 负责子任务注册与调度。这样的组织方式使系统功能分布较清楚，也便于后续单独替换模型网关、前端界面或 subagent 策略。

7.3 错误处理

系统在多个层次进行错误处理。API 层负责把请求校验错误、session 不存在、skill 名称非法等问题转换为明确的 HTTP 错误；模型调用层负责捕获连接失败、网关返回错误、模型输出为空和 JSON 解析失败等问题；前端则在 task pane 中展示错误信息，避免用户误以为系统已经成功完成修改。

尤其需要注意的是，模型输出必须经过 schema 校验后才能进入前端 action 渲染流程。如果模型返回非法 JSON 或不符合 TaskResponse 结构，后端会拒绝该结果，而不是让前端尝试猜测其含义。这一策略牺牲了一部分容错性，但能显著降低错误输出直接修改文档的风险。

7.4 安全与用户控制

项目的安全设计主要围绕本地密钥管理和文档修改控制展开。真实 API key 不应写入代码仓库，而是保存在本地配置文件或用户数据目录中。对于文档内容，系统默认作为本地开发型应用运行，后端和插件均在本机环境中工作，减少不必要的暴露。

在用户控制方面，所有可能修改文档的模型输出都通过 action 表达，并包



含原文、替换文本、修改理由、风险等级和确认标记。前端在应用修改前展示预览，并保存最近一次修改用于 **undo** 和 **review**。该设计使系统从“自动替用户改文档”转变为“向用户提交可审查的编辑建议”，更适合学术写作场景。



第八章 实验与评估

8.1 评估目标

本项目的评估重点不是比较不同大语言模型的生成质量，而是验证系统框架是否能够稳定支撑真实文档写作流程。因此，评估目标分为两类：第一类是工程正确性，即 API、schema、context builder、session、memory、subagent 和 action 数据流是否能够按预期运行；第二类是写作质量，即模型输出是否语义保真、语法正确且适合学术表达。当前项目主要完成第一类评估，第二类需要真实模型调用和人工评分，作为后续工作继续完善。

8.2 Subagent 单元测试

项目使用 `tests/test_subagents.py` 对 subagent 机制进行单元测试。测试采用 mock AI client 隔离模型随机性，重点验证后端调度逻辑本身是否正确。测试覆盖内容包括：未指定 subagent 时走普通 agent 流程、单个 subagent 能正确注入 skill、多个 subagent 的 action 和 final text 能被合并、自定义 subagent 能够运行、auto planner 能生成并过滤 subagent call，以及 stepwise run/merge endpoint 能够正常工作。

测试结果如表 8.1 所示。

表 8.1 Subagent 单元测试结果

Table 8.1 Subagent unit test results

测试对象	验证内容	结果
普通 agent flow	不启用 subagent 时的兼容路径	通过
单个 subagent	skill 注入和 action 返回	通过
多个 subagent	结果合并与 final text 选择	通过
自动规划	planner 生成和过滤 subagent call	通过
分步执行	run/merge endpoint 可用性	通过

完整测试结果为 11 passed, 1 warning。该结果说明 subagent 不是停留在概念层面的设计，而是已经具备完整执行路径和可验证行为的软件模块。



8.3 固定 Agent Flow 测试

固定流程测试使用 `scripts/test_agent_flow.py -mock` 验证从服务状态检查到 session 清理的端到端流程。测试依次调用健康检查、上下文构建、创建 session、写入 memory、发送 agent 消息、读取历史消息和删除 session 等接口。输入文本为一个简短的方法部分句子：

```
1 This method is good and useful.
```

测试结果显示，context builder 能正确保留选中文本并截取 before/after 上下文；mock agent 能返回中文解释、英文改写、`replace_selection` action 和低风险标记；消息存储和清理流程也能正常完成。该测试验证了 context、memory、agent response、message storage 和 cleanup 的完整链路。

8.4 多轮场景测试

多轮场景测试使用 `scripts/agent_scenario_test.py -mock` 和预设中文场景文件运行三轮对话。第一轮用户要求解释选中句子的问题并给出正式英文改写；第二轮要求基于上一轮解释给出更简洁版本；第三轮强调不要改变 scientific meaning，并要求再次生成版本。

测试通过检查每轮消息数量增长和响应 schema 合法性来验证多轮状态管理。结果显示，保存的消息数量按 2 -> 4 -> 6 增长，说明 session 能持续保存 user/assistant 消息，memory 和 document context 能在每轮请求中继续参与 prompt 构建。虽然 mock 输出不能代表真实模型质量，但它验证了多轮写作系统的工程基础。

8.5 消融式分析

除自动化测试外，本文还从系统设计角度分析关键模块的必要性。表 8.2 总结了若移除关键设计后可能出现的问题。

该分析说明，最终架构并非简单功能堆叠，而是围绕文档编辑场景中的上下文、控制、安全和可测试性逐步形成的设计。

表 8.2 关键设计的消融式分析

Table 8.2 Ablation-style analysis of key design choices

设计	移除后的预期问题
Context builder	不同客户端各自拼接上下文，选区重复或 offset 不一致时行为不可控
TaskResponse schema	前端无法稳定区分解释文本、最终文本和可执行 action
用户确认字段	模型输出可能直接覆盖文档，缺少用户审查环节
Session storage	多轮任务无法读取历史，也无法保存 assistant response
Session memory	写作目标、术语和用户偏好需要每轮重复输入
Subagent	复合任务全部挤入单一 prompt，容易造成职责混淆和过度改写
Runtime config	打包后端口变化时，插件仍依赖写死的 API 地址



第九章 讨论

9.1 有效设计

从系统实现和测试结果看，本项目较为有效的设计首先体现在“模型输出”与“文档编辑动作”的分离。后端并不直接修改 Word 文档，而是返回解释文本、最终改写结果和结构化 action；前端再负责展示、确认、应用与撤销。这一机制使模型建议始终处在人类可审查的流程中，降低了误改文档的风险，也更符合学术写作场景中“辅助修改”而非“自动替代”的使用方式。

其次，统一的 context builder 降低了不同客户端之间的接口差异。Word Add-in、命令行测试和 Web demo 都可能提供不同粒度的输入，但后端会将其规范化为一致的文本请求结构，再进入 prompt 构造与模型调用流程。这样可以避免上下文拼接逻辑分散在多个前端入口中，也使会话、记忆和 subagent 调度能够复用同一套数据基础。

第三，subagent 采用了轻量化实现，而不是依赖复杂的外部 agent 框架。系统在现有 FastAPI 服务层中实现 registry、planner、skill injection、allowed actions 与结果合并，使各类子任务能够按职责拆分，同时保持较强的可控性和可测试性。第九章中的单元测试也表明，该设计并非停留在概念层面，而是已经具备明确的执行路径。

9.2 局限性

当前系统仍存在若干限制。首先，自动化测试主要验证工程链路是否正确，尚不能充分说明真实模型输出的写作质量。语法修正、学术润色、术语保真和公式表达是否达到用户预期，还需要引入真实模型调用、人工评分和更系统的测试集。

其次，系统对 Word 文档结构的利用仍然有限。目前上下文主要来自 plain text、selection 和正文片段，尚未充分解析 heading、comment、footnote、reference、table 和 equation object 等结构化对象。因此，在长文档、复杂表格或包含大量公式的论文中，系统对局部语境和文档层级的理解仍可能不足。

此外，公式解析和 section scope 仍处于较基础阶段。公式处理主要覆盖常见 LaTeX 结构，对复杂公式、混合文本公式和 Word 原生公式对象的支持还不完整；section scope 尚未真正实现，系统在相关请求中仍会退回到当前选区或有



限上下文。最后，当前项目主要面向本地开发与课程展示，尚未完成面向普通用户的安装、证书信任、自动更新和安全发布流程。

9.3 经验总结

本项目的的主要经验是，构建可用的 LLM 写作助手不能只依赖 prompt 设计。Prompt 决定模型如何理解任务，但系统能否真正进入文档编辑流程，还取决于结构化 schema、上下文构造、状态管理、错误处理和用户控制机制。缺少 schema 时，前端难以稳定区分解释、改写文本和可执行动作；缺少 session memory 时，系统难以支持连续写作；缺少预览和撤销机制时，即使模型输出质量较高，也不适合直接修改用户文档。

因此，本项目的价值不只在接入大语言模型，而在于将模型能力嵌入一个可控的软件系统中。对 Word 写作助手而言，可靠的工程边界、明确的用户确认流程和可测试的数据流，与模型生成能力同样重要。这也说明，面向真实办公软件的 AI 应用更接近系统工程问题，而不是单一的文本生成问题。



第十章 未来工作

尽管当前系统已经实现了从 Word Add-in 到后端 agent 服务的基本闭环，但距离面向真实用户的稳定写作工具仍有进一步完善空间。后续工作可以从以下几个方向展开。

首先，需要建立更系统的真实质量评估。当前测试主要验证 API、schema、session、memory 和 subagent 调度等工程链路，尚未充分衡量模型在语法修正、学术润色、术语保真和公式表达中的实际效果。后续可以构建包含多类错误和多种写作任务的测试集，并引入多人评分，从准确性、可读性、学术风格和语义保真等维度评价系统输出。

其次，可以增强对 Word 文档结构的解析能力。现有系统主要依赖选区、正文文本和局部上下文，尚未充分利用 heading、table、caption、comment、footnote、reference 和 equation object 等结构化信息。若能够将这些对象纳入 context builder，系统将更准确地理解论文层级、表格含义、引用关系和公式语境，从而更适合处理长篇学术文档。

第三，需要提升 action 定位和冲突检测的可靠性。当前 action 已经能够携带 anchor text、occurrence 和替换文本，但前端主要仍围绕当前选区执行修改。后续可以实现更细粒度的 range-level 定位，在应用修改前检测原文是否发生变化，并在发生冲突时要求用户重新确认。这对于多人协作编辑或长时间会话尤为重要。

第四，可以探索 memory 的半自动更新机制。当前 memory 主要由用户显式维护，能够保存写作目标、术语偏好和风格要求。未来可以让模型在对话过程中提出 memory update proposal，再由用户确认是否写入。这样既能减少重复输入，也能避免模型自动记忆不可靠或敏感的信息。

最后，系统需要补齐部署与安全发布能力。当前项目更适合作为本地开发和课程展示系统，尚未完整处理安装器、证书信任、自动更新、CORS 限制、敏感数据保护和日志脱敏等问题。若要面向普通用户发布，这些工程问题需要与模型能力同等重视。



第十一章 结论

本文围绕 Microsoft Word 中的真实写作场景，设计并实现了一个可控的大语言模型写作助手系统 Word AI Assistant。系统以 Word Add-in 作为用户入口，以 FastAPI 后端作为统一服务层，结合 context builder、model gateway、session storage、memory、structured action 和 subagent dispatch，形成了从文档上下文读取到模型响应生成，再到前端预览、应用和撤销修改的完整流程。

与简单的 prompt 调用式工具相比，本系统的核心特点在于将模型能力嵌入可验证的软件结构中。后端通过统一 schema 约束模型输出，使前端能够稳定区分解文本、最终改写和可执行动作；context builder 负责规范化文档上下文，降低不同客户端输入差异；session 与 memory 支持多轮写作；subagent 机制则为语法、措辞、风格、公式和复合任务提供了可扩展的任务拆分方式。

实验与测试结果表明，当前系统的关键工程路径已经具备可运行性和可验证性。Subagent 单元测试、固定 agent flow 测试和多轮场景测试均验证了系统在上下文构造、消息存储、结构化响应、动作返回和子任务合并方面的基本正确性。虽然真实模型质量评估、Word 深层结构解析和部署安全仍需继续完善，但本项目已经展示了将通用 LLM 转化为文档写作工具所需的关键设计。

总体而言，Word AI Assistant 的价值不只在于能够调用大语言模型，而在于提供了一套面向文档编辑场景的可控、可扩展、可测试的写作辅助框架。该框架强调用户确认、结构化输出和工程边界，为后续构建更成熟的学术写作辅助系统奠定了基础。



附录 A API 请求示例

A.1 语法修改任务

以下示例展示了一个直接文本任务请求。客户端提供待修改文本、局部上下文和附加指令，后端将其转换为统一的 **TextRequest** 并进入任务执行流程。

```
1 {  
2   "text": "He dont know what to did yesterday.",  
3   "context": {  
4     "before": "",  
5     "after": ""  
6   },  
7   "instruction": ""  
8 }
```

A.2 带文档上下文的 Agent 对话

Agent 模式下，请求不仅包含用户消息，还包含选区、前后文、正文片段和当前作用范围。该结构使后端能够在多轮对话中复用文档上下文、历史消息和用户记忆。

```
1 {  
2   "message": "Explain the selected sentence and rewrite it formally.",  
3   "document_context": {  
4     "selection": {  
5       "text": "This method is good and useful.",  
6       "start": 120,  
7       "end": 156  
8     },  
9     "before": "We evaluate the proposed approach.",  
10    "after": "The result is reported in Table 1.",  
11    "body_text": "Full document text or excerpt.",  
12    "active_scope": "selection",  
13    "context_window_chars": 300,  
14    "instruction": "Keep the final rewrite suitable for an academic methods section."  
15  }  
16 }
```



A.3 显式 Subagent 调度

当用户明确要求多个子任务时，前端或后端 planner 可以指定 subagent 列表。系统会分别执行对应能力，再将解释、改写文本和 action 合并为统一响应。

```
{  
2  "message": "Check grammar and improve academic tone.",  
   "selection": {  
4    "text": "This method is good and useful."  
   },  
6  "subagents": ["proofread", "academic_polish"]  
}
```



附录 B 验证命令

本项目使用自动化测试和 mock 运行脚本验证核心工程链路。Subagent 单元测试命令如下：

```
1 $tmp = Join-Path (Get-Location) '.tmp\pytest'
   New-Item -ItemType Directory -Force -Path $tmp | Out-Null
3 $env:TMP = $tmp
   $env:TEMP = $tmp
5 pytest -q
```

测试结果为 11 passed, 1 warning。固定 agent flow 测试命令如下：

```
1 python .\scripts\test_agent_flow.py --mock
```

测试结果为 Agent flow test passed。多轮场景测试命令如下：

```
1 python .\scripts\agent_scenario_test.py --mock
```

测试结果为 Scenario test passed，并观察到保存消息数量按 2 -> 4 -> 6 增长。



附录 C 端到端运行流程

一次典型的 Word agent 编辑请求可概括为以下步骤：

1. 用户在 Word 文档中选中待修改文本。
2. 用户打开 Word AI 的 Agent task pane。
3. 前端通过 Office.js 读取 selection 和 body text。
4. 前端创建或复用后端 session。
5. 前端将设置页中的 memory 写入 session。
6. 前端发送 message 和 document context。
7. 后端调用 context builder 构造统一文本请求。
8. 后端读取历史消息和用户记忆。
9. 若启用自动子任务规划，后端先生成 subagent 调度计划。
10. 后端运行主 agent 或多个 subagent。
11. 后端合并结果并保存 assistant message。
12. 前端渲染 reply、summary、final text 和 actions。
13. 用户预览 action。
14. 用户点击 Apply，前端将修改写回 Word 文档。
15. 前端保存 lastApplied，用于后续撤销和对比查看。



致谢

感谢马智炜教授与本课助教在大模型API申请事宜上的鼎力协助。